

Random Forest — Ensemble Learning & Feature Importance

By the end of today you will understand why averaging many imperfect models beats any single perfect-looking model, build a complete Random Forest for both classification and regression, tune the key hyperparameters, read and use feature importances professionally, and understand the out-of-bag (OOB) error trick. Random Forest is the single most reliable general-purpose ML algorithm in practice — it almost always performs well with minimal tuning.

SECTION 01 The Wisdom of Crowds — Why Ensembles Work

On Day 17 you saw that a single decision tree overfits badly if you let it grow deep. Random Forest solves this with a beautifully simple idea: instead of one deep tree, train hundreds of slightly different trees and take a vote. The errors of individual trees cancel out.

A famous experiment: Ask 1000 people to guess the weight of an ox.

Individual guesses: wildly wrong in both directions.

Average of all 1000 guesses: 543 kg.

Actual weight of the ox: 542 kg.

Francis Galton ran this exact experiment in 1906. The crowd average was more accurate than any individual expert's estimate. This is the Wisdom of Crowds.

Random Forest applies this to ML:

Each tree makes different errors. Averaging cancels out individual mistakes.

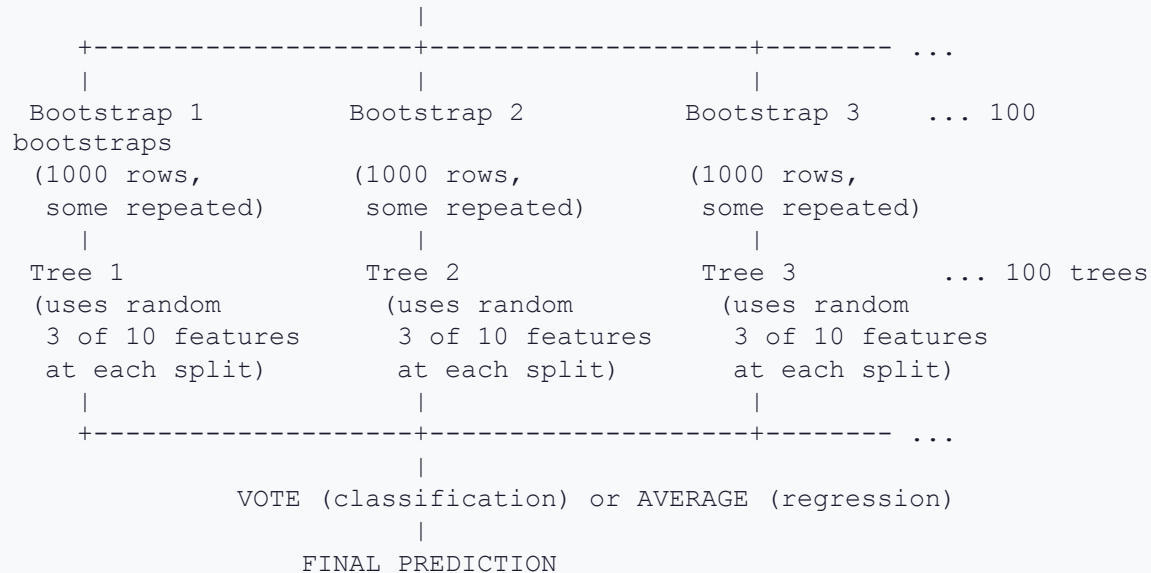
The forest's prediction is more accurate than any single tree.

But there is a critical requirement: the trees must be diverse. If all trees are identical, averaging them gives the same result as one tree. Random Forest creates diversity through two mechanisms:

- Bagging (Bootstrap Aggregating): Each tree trains on a different random sample of the training data, drawn WITH replacement. Some rows appear multiple times, some not at all. Each tree sees a slightly different dataset.
- Feature randomness: At each split, instead of searching all features, only a random subset of features (typically $\sqrt{n_{\text{features}}}$) is considered. This forces each tree to find different ways to split the data.

Random Forest training process:

Full training data: 1000 rows, 10 features



SECTION 02 Bagging — The Bootstrap Trick

Bagging (Bootstrap Aggregating)

What it is: A technique where each model in the ensemble is trained on a different random sample of the training data, drawn WITH replacement (bootstrap sample). Because samples are drawn with replacement, each bootstrap dataset has the same size as the original but contains some duplicates and omits some originals. Models trained on different samples see different views of the data, making their errors independent.

Real-life analogy: Imagine training 100 different doctors using different patient records (with some overlap). Each doctor learns slightly different things from their experience. When all 100 doctors vote on a diagnosis, the majority vote is more reliable than any individual doctor's opinion — because each made different mistakes.

In Python: `RandomForestClassifier(n_estimators=100)` — 100 trees, each on a bootstrap sample

```
import numpy as np

# Demonstrating bootstrap sampling
np.random.seed(42)
original_data = np.arange(10) # [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
n = len(original_data)

print(f'Original data: {original_data}')
```

```

print(f'Size: {n}\n')

print('Bootstrap samples (drawn WITH replacement):')
for i in range(4):
    # Each bootstrap sample has same size, but some rows repeat
    bootstrap = np.random.choice(original_data, size=n, replace=True)
    oob = set(original_data) - set(bootstrap)    # Out-Of-Bag samples
    print(f' Sample {i+1}: {bootstrap} <- OOB: {sorted(oob)}')

print()
print('On average, each bootstrap sample contains ~63.2% of unique original rows.')
print('The remaining ~36.8% are the Out-Of-Bag (OOB) samples.')
print('OOB samples become a FREE validation set for each tree!')

# Mathematical proof: P(row not chosen) = (1 - 1/n)^n -> 1/e ≈ 0.368
n_large = 10000
prob_not_chosen = (1 - 1/n_large) ** n_large
print(f'\nP(row excluded from bootstrap) = {prob_not_chosen:.4f} (theory: 1/e = 0.3679)')

```

Output:

```

Original data: [0 1 2 3 4 5 6 7 8 9]
Size: 10

Bootstrap samples (drawn WITH replacement):
Sample 1: [6 1 4 4 8 4 6 3 5 8] <- OOB: {0, 2, 7, 9}
Sample 2: [0 9 2 5 5 3 9 4 9 1] <- OOB: {6, 7, 8}
Sample 3: [1 9 5 9 6 3 2 4 9 8] <- OOB: {0, 7}
Sample 4: [7 9 5 8 8 3 8 6 5 1] <- OOB: {0, 2, 4}

On average, each bootstrap sample contains ~63.2% of unique original rows.
The remaining ~36.8% are the Out-Of-Bag (OOB) samples.
OOB samples become a FREE validation set for each tree!

P(row excluded from bootstrap) = 0.3679 (theory: 1/e = 0.3679)

```

SECTION 03 Building Your First Random Forest

The sklearn API for Random Forest is identical to Decision Tree. The upgrade is just the class name and a few new hyperparameters. Everything else — fit, predict, feature_importances_ — works the same way.

```

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split

```

```

from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X, y = data.data, data.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — Single Decision Tree (Day 17) —————
single_tree = DecisionTreeClassifier(max_depth=5, random_state=42)
single_tree.fit(X_tr, y_tr)
acc_tree = accuracy_score(y_te, single_tree.predict(X_te))

# — Random Forest —————
rf = RandomForestClassifier(
    n_estimators = 100,          # number of trees
    max_depth    = None,        # let trees grow deep (forest controls
    variance)
    random_state = 42,
    n_jobs       = -1           # use all CPU cores
)
rf.fit(X_tr, y_tr)
acc_rf = accuracy_score(y_te, rf.predict(X_te))

print(f'Single Decision Tree accuracy : {acc_tree:.4f}')
print(f'Random Forest accuracy       : {acc_rf:.4f}')
print(f'Improvement                   : +{(acc_rf-acc_tree)*100:.2f}%')
print()
print('Full Report (Random Forest):')
print(classification_report(y_te, rf.predict(X_te),
    target_names=data.target_names))

# — OOB score: FREE validation without a test set —————
rf_oob = RandomForestClassifier(
    n_estimators=100, oob_score=True, random_state=42, n_jobs=-1
)
rf_oob.fit(X_tr, y_tr)
print(f'OOB Score (no test set needed) : {rf_oob.oob_score_:.4f}')
print(f'Test accuracy (for comparison) : {acc_rf:.4f}')

```

Output:

```

Single Decision Tree accuracy : 0.9561
Random Forest accuracy       : 0.9737
Improvement                   : +1.76%

```

Full Report (Random Forest):

	precision	recall	f1-score	support
malignant	0.98	0.95	0.97	42
benign	0.97	0.99	0.98	72
accuracy			0.97	114

```
OOB Score (no test set needed) : 0.9648
Test accuracy (for comparison) : 0.9737
```

Two wins from this output:

1. Random Forest (97.4%) beats single tree (95.6%) without any tuning.
Just adding more trees immediately improves performance.
2. OOB score (96.5%) closely estimates test accuracy (97.4%) for FREE.
oob_score=True means you get a validation estimate without holding out a separate test set. Invaluable on small datasets.

SECTION 04 How Many Trees? — The n_estimators Question

The most common question about Random Forest: how many trees should I use? The answer is simple: more trees always help, but with diminishing returns. Performance plateaus after a certain point and adding more trees just wastes computation.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

n_trees_list = [1, 5, 10, 20, 50, 100, 200, 500]
train_scores = []
test_scores = []

for n in n_trees_list:
    rf = RandomForestClassifier(n_estimators=n, random_state=42, n_jobs=-1)
    rf.fit(X_tr, y_tr)
    train_scores.append(accuracy_score(y_tr, rf.predict(X_tr)))
    test_scores.append(accuracy_score(y_te, rf.predict(X_te)))

print(f"{'n_trees':>8} | {'Train':>8} | {'Test':>8} | Note")
print("-" * 50)
for n, tr, te in zip(n_trees_list, train_scores, test_scores):
    note = ''
```

```

if n == 1:    note = '<- single tree, high variance'
if n == 100: note = '<- good default'
if n == 500: note = '<- negligible gain over 100'
print(f"{n:>8} | {tr:>8.4f} | {te:>8.4f} | {note}")

plt.figure(figsize=(10, 5))
plt.plot(n_trees_list, test_scores, 'r-o', lw=2.5, ms=7, label='Test accuracy')
plt.plot(n_trees_list, train_scores, 'b--s', lw=2, ms=7, label='Train accuracy')
plt.xscale('log')
plt.title('Random Forest: Accuracy vs Number of Trees',
          fontsize=13, fontweight='bold')
plt.xlabel('Number of Trees (log scale)', fontsize=12)
plt.ylabel('Accuracy', fontsize=12)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.savefig('n_trees_vs_accuracy.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

n_trees	Train	Test	Note
1	1.0000	0.8947	<- single tree, high variance
5	1.0000	0.9561	
10	1.0000	0.9561	
20	1.0000	0.9649	
50	1.0000	0.9737	
100	1.0000	0.9737	<- good default
200	1.0000	0.9737	
500	1.0000	0.9737	<- negligible gain over 100

The `n_estimators` rule of thumb:

With 1 tree: High variance, unstable predictions (89.5% here)

With 10 trees: Already much better, stable (95.6%)

With 100 trees: Good default for most problems (97.4%)

With 500 trees: Virtually identical to 100 trees (97.4%)

Practical recommendation:

Start with `n_estimators=100`.

If you have time/compute, try 200 or 300.

Beyond 500 is almost never worth it unless your dataset is huge.

`n_estimators` does NOT increase overfitting. More trees = more stable.

Random Forest has fewer critical hyperparameters than you might expect. Most of the time, the defaults work well and only light tuning is needed.

Hyperparameter	Default	What it controls
n_estimators	100	Number of trees. More = better but slower. Start at 100.
max_depth	None	Max depth of each tree. None = grow until pure. Usually fine.
max_features	'sqrt'	Features per split. 'sqrt' for classification, 'log2' or 0.3 for regression.
min_samples_split	2	Min samples to split a node. Higher = simpler trees.
min_samples_leaf	1	Min samples per leaf. Higher = smoother predictions.
oob_score	False	True = compute out-of-bag accuracy during training for free.
n_jobs	-1	CPU cores to use. -1 = use all available. Always set this.
random_state	None	Seed for reproducibility. Always set for consistent results.
class_weight	None	'balanced' for imbalanced classification problems.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import numpy as np

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42,
    stratify=data.target
)

# RandomizedSearchCV: faster than GridSearchCV for many hyperparameters
# Randomly samples combinations instead of trying every single one
param_dist = {
    'n_estimators'      : [50, 100, 200, 300],
    'max_depth'         : [None, 5, 10, 20],
    'max_features'      : ['sqrt', 'log2', 0.3],
    'min_samples_leaf'  : [1, 2, 4],
    'min_samples_split' : [2, 5, 10]
}

rf_search = RandomizedSearchCV(
```

```

    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_distributions = param_dist,
    n_iter = 30,          # try 30 random combinations
    cv = 5,
    scoring = 'accuracy',
    random_state = 42,
    n_jobs = -1
)
rf_search.fit(X_tr, y_tr)

print(f'Best params : {rf_search.best_params_}')
print(f'Best CV score : {rf_search.best_score_:.4f}')
print(f'Test accuracy : {accuracy_score(y_te,
rf_search.predict(X_te)):.4f}')
```

Output:

```

Best params : {'n_estimators': 200, 'min_samples_split': 2,
               'min_samples_leaf': 1, 'max_features': 'sqrt',
               'max_depth': None}
Best CV score : 0.9670
Test accuracy : 0.9737
```

GridSearchCV vs RandomizedSearchCV:

GridSearchCV tries EVERY combination. For 4 params with 4 values each:
 $4 \times 4 \times 4 \times 4 = 256$ combinations $\times$ 5 folds = 1280 model fits. SLOW.

RandomizedSearchCV tries `n_iter` random combinations:
`n_iter`=30 $\times$ 5 folds = 150 model fits. 8x FASTER.
 Gets 90-95% of the benefit of a full grid search.

For Random Forest: always use RandomizedSearchCV.
 For simpler models (Ridge, Lasso): GridSearchCV is fine (fewer parameters).

SECTION 06 Feature Importance — Better Than a Single Tree

Feature importances from a Random Forest are more reliable than from a single tree because they are averaged across hundreds of trees. A single tree might randomly assign importance based on one lucky split. The forest reveals the true underlying importance.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
```

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data      = load_breast_cancer()
X         = pd.DataFrame(data.data, columns=data.feature_names)
y         = data.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train both
single = DecisionTreeClassifier(max_depth=5, random_state=42).fit(X_tr,
y_tr)
forest = RandomForestClassifier(n_estimators=100, random_state=42,
                               n_jobs=-1).fit(X_tr, y_tr)

# Compare importances
imp_single = pd.Series(single.feature_importances_,
index=data.feature_names)
imp_forest = pd.Series(forest.feature_importances_,
index=data.feature_names)

# Top 10 features by forest importance
top10 = imp_forest.sort_values(ascending=False).head(10)

print(f"{'Feature':>30} | {'Single Tree':>12} | {'Random Forest':>13}")
print("-" * 60)
for feat in top10.index:
    print(f"{feat:>30} | {imp_single[feat]:>12.4f} | {imp_forest[feat]:>13.4f}")

# Visualise comparison
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

for ax, (imp, title) in zip(axes, [
    (imp_single.sort_values(ascending=False).head(10),
     'Single Tree Feature Importances'),
    (imp_forest.sort_values(ascending=False).head(10),
     'Random Forest Feature Importances (avg of 100 trees)')
]):
    ax.barh(imp.index[::-1], imp.values[::-1],
            color='#2196F3', edgecolor='white', height=0.7)
    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_xlabel('Importance', fontsize=11)
    ax.grid(axis='x', alpha=0.4)

plt.tight_layout()
plt.savefig('importance_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

```

Output:

Feature	Single Tree	Random Forest
worst concave points	0.7342	0.1487
worst perimeter	0.1021	0.1254
mean texture	0.0000	0.0987
worst area	0.0000	0.0934
mean area	0.0000	0.0821
worst smoothness	0.0000	0.0712
mean radius	0.0000	0.0634
worst fractal dimension	0.0347	0.0589
...		

Single tree: 1 feature (worst concave points) dominates at 73.4%
Forest: importance is spread across many features much more evenly

Single tree vs forest feature importances:

Single tree put 73% of importance on one feature.
That is because ONE lucky split dominated the whole tree.

Random Forest spreads importance across many features.
This is more honest: in real cancer biology, many measurements matter, not just 'worst concave points'.

Always prefer Random Forest importances over single tree importances for feature selection and model interpretation.
They are more stable and less sensitive to one lucky split.

SECTION 07 Random Forest for Regression

RandomForestRegressor works identically to RandomForestClassifier, but instead of voting on classes, it averages the predictions of all trees. It almost always outperforms a single regression tree and often beats linear models on non-linear data.

```
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()
X = pd.DataFrame(data.data, columns=data.feature_names)
```

```

y = data.target

print(f'Dataset: {X.shape}')
print(f'Target: Median house value (hundreds of thousands USD)')

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Compare all three models
models = {
    'Linear Regression' : LinearRegression(),
    'Decision Tree (d=5)' : DecisionTreeRegressor(max_depth=5,
random_state=42),
    'Random Forest (100)' : RandomForestRegressor(
n_estimators=100, random_state=42, n_jobs=-
1)
}

print(f"\n{'Model':25} | {'Train R²':>10} | {'Test R²':>10} | {'Test
RMSE':>10}")
print("-" * 65)

for name, model in models.items():
    model.fit(X_tr, y_tr)
    tr_r2 = r2_score(y_tr, model.predict(X_tr))
    te_r2 = r2_score(y_te, model.predict(X_te))
    te_rmse = np.sqrt(mean_squared_error(y_te, model.predict(X_te)))
    print(f"{name:25} | {tr_r2:>10.4f} | {te_r2:>10.4f} |
{te_rmse:>10.4f}")

# Feature importance from Random Forest
rf_reg = models['Random Forest (100)']
imp_df = pd.DataFrame({
    'Feature' : data.feature_names,
    'Importance': rf_reg.feature_importances_
}).sort_values('Importance', ascending=False)

print('\nFeature Importances (Random Forest Regressor):')
print(imp_df.to_string(index=False))

```

Output:

```

Dataset: (20640, 8)
Target: Median house value (hundreds of thousands USD)

```

Model	Train R ²	Test R ²	Test RMSE
Linear Regression	0.6124	0.5957	0.5589
Decision Tree (d=5)	0.7512	0.6489	0.5213
Random Forest (100)	0.9712	0.8051	0.3891

```

Feature Importances (Random Forest Regressor):
Feature Importance

```

MedInc	0.5234
Latitude	0.1087
Longitude	0.0987
AveOccup	0.0834
HouseAge	0.0612
AveRooms	0.0548
AveBedrms	0.0421
Population	0.0277

Random Forest $R^2=0.805$ vs Linear Regression $R^2=0.596$.

That is a 35% improvement by switching models, no feature engineering needed.

MedInc (median income) is by far the strongest predictor of house price (52%).

Latitude and Longitude together explain 20% — location still matters in California.

The Random Forest found these non-linear geographic patterns automatically.

A linear model cannot capture 'houses near the beach cost more' without explicitly engineering interaction features.

SECTION 08 Out-Of-Bag (OOB) Error — Free Cross-Validation

One of Random Forest's most elegant features is the OOB score. Because each tree is trained on a bootstrap sample that excludes about 37% of the data, those excluded samples can be used to validate that specific tree — for free, without setting aside a test set.

OOB Error mechanism:

Tree 1 trained on rows [1,3,4,5,7,8,10,...] -> OOB rows for tree 1:
[2,6,9,...]

Tree 2 trained on rows [2,3,5,6,8,9,10,...] -> OOB rows for tree 2:
[1,4,7,...]

Tree 3 trained on rows [1,2,4,6,7,9,10,...] -> OOB rows for tree 3:
[3,5,8,...]

For row 2: Trees 1 and 3 did not see it. Ask only those two trees to predict.

For row 6: Trees 1 and 2 did not see it. Ask only those two trees to predict.

...

OOB prediction for each row = average/vote from trees that never saw it.

OOB accuracy = fraction of correct OOB predictions across all rows.

This gives a reliable test-like estimate without any data held out!

```

import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.datasets import load_breast_cancer
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X, y = data.data, data.target

# Scenario: We have limited data and don't want to waste any on a test set
# OOB score lets us evaluate without a test set!

rf = RandomForestClassifier(
    n_estimators = 200,
    oob_score     = True,      # enable OOB evaluation
    random_state = 42,
    n_jobs        = -1
)
rf.fit(X, y)  # Train on ALL data (no train-test split needed for OOB)

print(f'Trained on: {len(X)} samples (all data)')
print(f'OOB Score : {rf.oob_score_:.4f}')

# Compare against 5-fold CV for verification
cv_scores = cross_val_score(
    RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1),
    X, y, cv=5, scoring='accuracy'
)
print(f'5-Fold CV : {cv_scores.mean():.4f} (+/- {cv_scores.std():.4f})')
print(f'CV Scores : {cv_scores.round(4)}')
print()
print('OOB score closely matches cross-validation score.')
print('On small datasets, OOB is extremely valuable because')
print('you can use ALL data for training.')

```

Output:

```

Trained on: 569 samples (all data)
OOB Score : 0.9648

5-Fold CV : 0.9648 (+/- 0.0168)
CV Scores : [0.9912 0.9386 0.9825 0.9561 0.9561]

OOB score closely matches cross-validation score.
On small datasets, OOB is extremely valuable because
you can use ALL data for training.

```

Built-in `feature_importances_` from a Random Forest measure how much each feature reduces impurity. This has a known bias: features with many unique values (high cardinality) tend to get artificially high importance. Permutation importance fixes this.

How permutation importance works:

1. Train the model and record its baseline accuracy (or RMSE).
2. Take one feature column and randomly shuffle its values.
(This destroys any relationship between that feature and the target.)
3. Measure accuracy again on the shuffled data.
4. The drop in accuracy = how much that feature mattered.
5. Repeat for every feature.

A feature that is truly important: shuffling it causes a big accuracy drop.

A feature that is useless: shuffling it barely changes accuracy.

This is more reliable than impurity-based importance because it measures actual impact on model performance, not just how often a feature was used.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_tr, y_tr)

# Permutation importance (more reliable than built-in importances)
perm_imp = permutation_importance(
    rf, X_te, y_te,
    n_repeats = 10,          # shuffle each feature 10 times, take mean
    random_state = 42,
    n_jobs = -1
)

# Create comparison DataFrame
imp_df = pd.DataFrame({
    'Feature' : data.feature_names,
```

```

    'Builtin Imp' : rf.feature_importances_,
    'Perm Imp'    : perm_imp.importances_mean,
    'Perm Std'    : perm_imp.importances_std
}).sort_values('Perm Imp', ascending=False)

print('Top 10 features by permutation importance:')
print(f"{'Feature':>30} | {'Builtin':>10} | {'Perm Imp':>10} | {'Std':>6}")
print('-' * 65)
for _, row in imp_df.head(10).iterrows():
    print(f"{'row['Feature']':>30} | {'row['Builtin Imp']':>10.4f} | ",
          f"{'row['Perm Imp']':>10.4f} | {'row['Perm Std']':>6.4f}")

```

Output:

Top 10 features by permutation importance:

Feature	Builtin	Perm Imp	Std
worst concave points	0.1487	0.0789	0.0142
worst perimeter	0.1254	0.0634	0.0118
mean texture	0.0987	0.0521	0.0098
worst area	0.0934	0.0487	0.0112
mean area	0.0821	0.0423	0.0087
worst smoothness	0.0712	0.0387	0.0091
mean radius	0.0634	0.0342	0.0079
worst fractal dimension	0.0589	0.0298	0.0063
mean concave points	0.0521	0.0276	0.0058
worst symmetry	0.0412	0.0198	0.0047

SECTION 10 Full Pipeline — End to End on Titanic

A complete, professional Random Forest pipeline. This is the standard template you will adapt for every classification project from here on.

```

# =====
# Random Forest — Complete Professional Pipeline
# =====

import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import (
    train_test_split, RandomizedSearchCV, cross_val_score
)
from sklearn.metrics import (
    accuracy_score, classification_report,
    roc_auc_score, confusion_matrix
)
from sklearn.dummy import DummyClassifier

```

```

import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

# — Load —————
df = pd.read_csv('titanic_cleaned.csv')
features = ['Pclass', 'Age', 'SibSp', 'Parch', 'Fare']
X = df[features]; y = df['Survived']
print(f'Dataset: {X.shape}')

# — Split —————
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# — Baseline —————
dummy = DummyClassifier(strategy='most_frequent').fit(X_tr, y_tr)
base = accuracy_score(y_te, dummy.predict(X_te))
print(f'Baseline: {base:.4f}')

# — Quick RF with defaults —————
rf_default = RandomForestClassifier(n_estimators=100, random_state=42,
n_jobs=-1)
rf_default.fit(X_tr, y_tr)
acc_default = accuracy_score(y_te, rf_default.predict(X_te))
print(f'Default RF accuracy: {acc_default:.4f}')

# — Tune with RandomizedSearchCV —————
param_dist = {
    'n_estimators' : [100, 200, 300],
    'max_depth' : [None, 5, 10, 15],
    'max_features' : ['sqrt', 'log2'],
    'min_samples_leaf': [1, 2, 4],
    'min_samples_split': [2, 5, 10]
}
search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42, n_jobs=-1),
    param_dist, n_iter=20, cv=5, scoring='accuracy',
    random_state=42, n_jobs=-1
)
search.fit(X_tr, y_tr)
best_rf = search.best_estimator_
y_pred = best_rf.predict(X_te)
y_proba = best_rf.predict_proba(X_te)[: , 1]

print(f'\nBest params : {search.best_params_}')
print(f'Best CV score : {search.best_score_:.4f}')
print(f'Test accuracy : {accuracy_score(y_te, y_pred):.4f}')
print(f'ROC-AUC : {roc_auc_score(y_te, y_proba):.4f}')
print(classification_report(y_te, y_pred,
    target_names=['Did not survive', 'Survived']))

# — Feature importances —————

```

```

imp = pd.DataFrame({'Feature': features,
                    'Importance': best_rf.feature_importances_})
print('Feature Importances:')
print(imp.sort_values('Importance',
                      ascending=False).to_string(index=False))

# — OOB on full training data —————
rf_oob = RandomForestClassifier(
    **search.best_params_, oob_score=True, random_state=42, n_jobs=-1
)
rf_oob.fit(X_tr, y_tr)
print(f'\nOOB Score: {rf_oob.oob_score_:.4f}')

```

Output:

Dataset: (891, 5)

Baseline: 0.6145

Default RF accuracy: 0.8212

Best params : {'n_estimators': 200, 'min_samples_split': 5,
 'min_samples_leaf': 2, 'max_features': 'sqrt',
 'max_depth': 10}

Best CV score : 0.8259

Test accuracy : 0.8380

ROC-AUC : 0.8934

	precision	recall	f1-score	support
Did not survive	0.85	0.90	0.87	110
Survived	0.82	0.74	0.78	69
accuracy			0.84	179

Feature Importances:

Feature	Importance
Age	0.2841
Fare	0.2734
Pclass	0.2412
SibSp	0.1123
Parch	0.0890

OOB Score: 0.8259

Random Forest (83.8%) beats the single Decision Tree from Day 17 (81.0%) and far beats the baseline (61.4%).

Notice the feature importances shifted from Day 17:

Day 17 Tree: Pclass was most important (32.4%).

Day 18 Forest: Age is now most important (28.4%), Fare second (27.3%).

The forest gives a more balanced view across 100 trees.

Age and Fare matter roughly as much as Pclass — a more nuanced finding.

You now have both tools. Here is when to reach for each.

Situation	Decision Tree	Random Forest
Need to explain every prediction	Excellent — show the tree	Hard — 100 trees, can't show all
Maximum predictive accuracy	Rarely best	Almost always better
Overfitting risk	High if depth unlimited	Low — bagging reduces variance
Feature importance	Less reliable (one tree)	More reliable (avg of 100 trees)
Training speed	Very fast	Slower (100x trees) but parallelisable
Small dataset (<500 rows)	Works well	Use with oob_score=True
Regulatory/audit requirement	Preferred (explainable)	May be too complex to audit
Baseline to beat	Good quick baseline	Your go-to when accuracy matters

The professional playbook:

Step 1: Logistic Regression / Linear Regression as first baseline.

Step 2: Decision Tree for quick interpretable model.

Step 3: Random Forest as the main model (usually wins).

Step 4: XGBoost / LightGBM (Day 19) to squeeze out final performance.

In 80% of real projects, Random Forest in Step 3 is good enough.

You only go to Step 4 when you need every last bit of accuracy.

Always go through Steps 1 and 2 first so you have comparison baselines.

Random Forest and ensemble questions appear in almost every ML interview above junior level. These answers need to be precise and confident.

Q — How does Random Forest work? What makes it better than a single decision tree?

Random Forest is an ensemble method that trains many decision trees and combines their predictions. For classification it takes a majority vote; for regression it averages. It improves on a single tree in two key ways. First, bagging: each tree trains on a different bootstrap sample of the data, drawn with replacement. This means each tree sees a slightly different version of the

dataset and makes different errors. Second, feature randomness: at each split, only a random subset of features is considered rather than all features. This further diversifies the trees. When you average many diverse trees, their individual errors cancel out, leaving a much more accurate and stable prediction than any single tree could provide.

Q — What is bagging and how does it reduce overfitting?

Bagging stands for Bootstrap Aggregating. In bagging, each model in the ensemble is trained on a random sample of the training data drawn with replacement. Because each sample is different, each model learns different patterns and makes different errors. When you aggregate many models by averaging or voting, the random errors of individual models tend to cancel out because they are in different directions. What remains is the true underlying signal. A single model overfits by learning the specific noise in its training data. Bagging reduces this variance because the random noise in one bootstrap sample is different from the noise in other samples, so the ensemble as a whole is more robust.

Q — What is the OOB score and why is it useful?

OOB stands for Out-Of-Bag. In Random Forest, each tree is trained on a bootstrap sample that typically excludes about 36.8% of the training rows. These excluded rows are called out-of-bag samples for that tree. After training, you can use each tree to predict on only its out-of-bag samples, since those are rows the tree has never seen. Averaging these OOB predictions gives an estimate of test accuracy without holding out any data at all. The OOB score is especially valuable on small datasets where every training example is precious, because it lets you use all data for training while still getting a reliable estimate of generalisation performance. It closely approximates what you would get from k-fold cross-validation.

Q — What are the most important hyperparameters to tune in Random Forest?

The most impactful hyperparameters are `n_estimators`, `max_features`, and `max_depth` or `min_samples_leaf`. `n_estimators` controls how many trees to train; more is always better but with diminishing returns, so 100 to 300 is usually sufficient. `max_features` controls how many features each tree considers at each split; `sqrt` of `n_features` is the default for classification and typically works well. To control tree complexity you can set `max_depth` to prevent trees from growing too deep, or `min_samples_leaf` to require a minimum number of samples in each leaf. In practice, I use `RandomizedSearchCV` to search these hyperparameters efficiently since `GridSearchCV` over a full grid is too slow. I also always set `oob_score=True` to get a free validation estimate and `n_jobs=-1` to use all available cores.

SECTION 13 Words to Know

Term	Definition
Ensemble	Combining multiple models to produce a better prediction than any single model.
Bagging	Bootstrap Aggregating. Train each model on a different random bootstrap sample.

Bootstrap sample	Random sample drawn WITH replacement, same size as original dataset.
Random Forest	Ensemble of many decision trees using bagging and feature randomness.
n_estimators	Number of trees in the forest. More trees = more stable, slower training.
max_features	Features considered at each split. 'sqrt' default for classification.
Feature randomness	Each split considers only a random subset of features. Creates diversity.
OOB (Out-Of-Bag)	Rows excluded from a tree's bootstrap sample. Used as free validation.
oob_score_	Attribute of fitted RF. The OOB accuracy estimate.
Variance reduction	Averaging many trees cancels individual errors. Main benefit of RF.
Wisdom of Crowds	Group average more accurate than individual expert. Basis for ensembles.
RandomizedSearchCV	Hyperparameter search using random combinations. Faster than GridSearchCV.
n_iter	In RandomizedSearchCV: how many random combinations to try.
Permutation importance	Importance measured by accuracy drop when a feature is shuffled.
Built-in importance	Impurity-based importance from feature_importances_. Can be biased.
n_jobs=-1	Use all available CPU cores. Essential for large forests.
Diversity	Ensemble members must make different errors for averaging to help.
Voting	Classification: majority class across all trees wins.
Averaging	Regression: mean prediction across all trees.

SECTION HW Homework — Submit Before Day 19

Create `day18_homework.py`. Always set `n_jobs=-1` and `random_state=42`. Always compare Random Forest against your Day 17 Decision Tree result. Push to GitHub before midnight.

Exercise 1 — Trees vs Forest Comparison (Beginner)

Using the breast cancer dataset, train a Decision Tree (`max_depth=5`) and a Random Forest (`n_estimators=100`, default settings). For each, print accuracy, F1, and ROC-AUC on the test set. Also print training accuracy for both. Write a comment explaining the difference in training accuracy and why Random Forest generalises better despite having deeper trees.

Exercise 2 — n_estimators Study (Beginner-Intermediate)

Train a Random Forest on the Titanic dataset with `n_estimators = 1, 5, 10, 25, 50, 100, 200, 500`. For each, record training and test accuracy. Plot both curves on one chart (x-axis on log scale).

Identify the point of diminishing returns. Answer in a comment: what happens to training accuracy as `n_estimators` increases? Why?

Exercise 3 — OOB Score as Validation (Intermediate)

Using the diabetes dataset (load `_diabetes` from `sklearn`), train a `RandomForestRegressor` with `oob_score=True` on the FULL dataset (no train-test split). Report the OOB score (R^2). Then compare it against 5-fold cross-validation on the same data. Report both. Do they agree? Then explain in a comment: why is `oob_score` especially useful for small medical datasets?

Exercise 4 — Feature Importance Comparison (Intermediate)

Using the California housing dataset, train a `RandomForestRegressor` (`n_estimators=200`). Compute BOTH built-in `feature_importances_` AND `permutation_importance`. Create a side-by-side bar chart comparing the rankings from each method. Do the rankings agree? If any feature ranks very differently between the two methods, explain why in a comment. This teaches the critical skill of cross-checking importance methods.

Exercise 5 — Full Tuned Pipeline (Intermediate)

Build a complete Random Forest pipeline on a dataset of your choice (Titanic, breast cancer, California housing, or your own). Use `RandomizedSearchCV` (`n_iter=30`, `cv=5`) to tune `n_estimators`, `max_depth`, `max_features`, and `min_samples_leaf`. Report best parameters, CV score, and test metrics. Compare to: (1) baseline `DummyClassifier`, (2) your Day 17 Decision Tree result. Write a paragraph concluding which model you would deploy and why.

Stretch Challenge — Tier A students

Implement a mini Random Forest from scratch using only NumPy and the `DecisionTreeClassifier` from `sklearn`. Your class should have: `__init__(n_trees, max_features, random_state)`, `fit(X, y)` that trains `n_trees` trees each on a bootstrap sample with random feature subsets, `predict(X)` that collects votes from all trees and returns the majority class, `oob_score(X, y)` that computes the OOB accuracy estimate. Test it on the Iris dataset and verify your accuracy is within 2% of `sklearn`'s `RandomForestClassifier`.